Diagramas de Flujo y Pseudocódigo

Sesión 2 — BIC01: Introducción a la Computación

Facultad de Ciencias
Escuela Profesional de Ciencia de la Computación
Universidad Nacional de Ingeniería

Tabla de Contenidos

1. Algoritmos y su Representación
2. Variables y Tipos de Datos en PSeInt
3. Operadores y Expresiones en PSeInt
4. Entrada y Salida de Datos en PSeInt
5. Funciones Matemáticas en PSeInt
6. Estructuras de Control
7. Variables Especiales: Contadores, Acumuladores e Indicadores
8. Lectura Complementaria: Arreglos y Funciones
9. Resumen

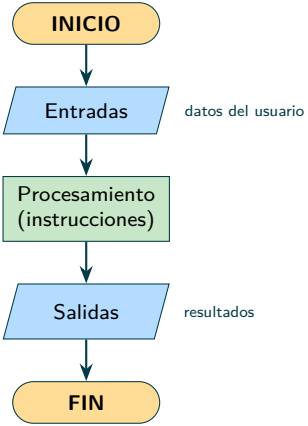
¿Qué es un Algoritmo? 1/4

Definición

Secuencia **ordenada y finita** de instrucciones para realizar una tarea o resolver un problema.

Propiedades fundamentales:

- **Finito**: termina en un número definido de pasos
- **Definido**: cada paso es claro y sin ambigüedad
- **Entradas**: recibe datos (0 o más)
- **Salidas**: produce al menos un resultado
- **Efectivo**: cada instrucción es ejecutable



Herramienta del curso

Usaremos **PSeInt** para diseñar y ejecutar algoritmos en pseudocódigo.

`https://pseint.sourceforge.net/`

Formas de Representar un Algoritmo 2/4

Pseudocódigo

Descripción compacta de un algoritmo usando **lenguaje natural estructurado**. Es independiente del lenguaje de programación.

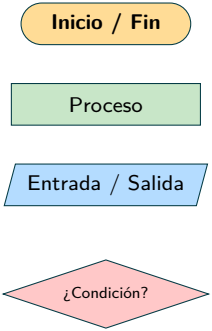
- ✔ Fácil de escribir y leer
- ✔ No requiere herramientas especiales
- ✔ Permite pensar la lógica antes de codificar
- ✔ Se usa PSeInt en el curso

Diagrama de Flujo

Representación **gráfica** de un algoritmo mediante símbolos estándar (normas **ANSI** e ISO).

- ✔ Visual e intuitivo
- ✔ Facilita detectar errores lógicos
- ✔ Útil para comunicar algoritmos
- ✔ PSeInt lo genera automáticamente

Símbolos Estándar del Diagrama de Flujo 3/4



Reglas importantes

- El flujo va de **arriba hacia abajo** y de **izquierda a derecha**
- Siempre comienza con **Inicio** y termina con **Fin**
- Cada símbolo tiene una función específica
- Las flechas indican la **dirección del flujo**
- Una decisión siempre tiene salida **Verdadero** y **Falso**

En PSeInt

El diagrama de flujo se genera **automáticamente** al escribir el pseudocódigo. ¡No necesitas dibujarlo manualmente!

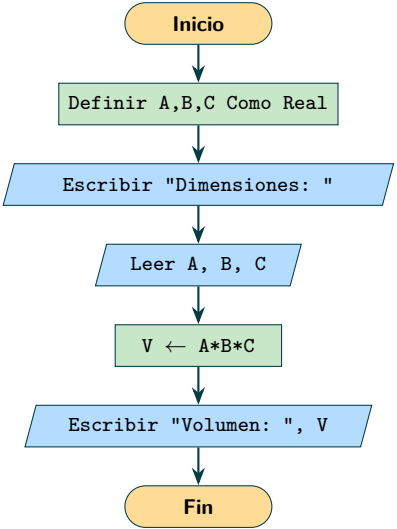
Ejemplo: Volumen de una Caja

Problema: Calcular el volumen de una caja conociendo sus tres dimensiones.

Pseudocódigo:

```
Algoritmo volumen_caja
  // Ingreso de datos
  Definir A, B, C Como Real
  Escribir 'Dimensiones (A,B,C): '
  Leer A, B, C

  // Procesamiento
  V <- A * B * C

  // Salida de datos
  Escribir 'Volumen: ', V
FinAlgoritmo
```

Diagrama de flujo:



Variables: ¿Qué son y para qué sirven?

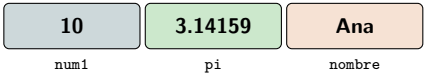
Definición

Una **variable** es un espacio en memoria con un nombre, que permite **almacenar y modificar** un valor durante la ejecución del algoritmo.

Tipos de datos en PSeInt:

Tipo	Alias	Ejemplo
Entero	—	5, -3, 100
Real	Numero	3.14, -0.5
Carácter	Texto, Cadena	"Hola"
Lógico	—	Verdadero, Falso

```
Algoritmo definiendo_variables
  Definir num1, num2 Como Entero
  Definir pi, area Como Real
  Definir nombre Como Caracter
  Definir esMayor Como Logico

  num1 <- 10
  pi   <- 3.14159
  nombre <- 'Ana'
  esMayor <- Verdadero

  Escribir num1, pi, nombre
FinAlgoritmo
```



Operadores en PSeInt

Operadores Relacionales		
>	Mayor que	3>2 → V
<	Menor que	1<0 → F
=	Igual que	4=4 → V
<=	Menor o igual	3<=3 → V
>=	Mayor o igual	4>=5 → F
<>	Distinto	'a'<>'b' → V

Operadores Lógicos		
Y / &	Conjunción (AND)	ambos V
O / |	Disyunción (OR)	uno es V
NO / ~	Negación (NOT)	invierte

Operadores Algebraicos		
+ - * /	Básicos	
^	Potencia	2^3 = 8
MOD/%	Módulo	7 MOD 2 = 1

Precedencia de Operadores

1. ( ) — paréntesis
2. ^ — potencia
3. *, /, MOD
4. +, -
5. =, >, <, <>, >=, <=
6. NO
7. Y
8. O

Ejemplo de expresión

2 + 3 * 4 = 14 ✔

(2 + 3) * 4 = 20 ✔

10 MOD 3 = 1 ✔

Operador de Asignación (<-) en PSeInt 2/2

¿Cómo funciona?

El operador **<-** asigna el valor del lado derecho a la variable del lado izquierdo.

```
num1 <- 4          // num1 vale 4
num2 <- 3          // num2 vale 3
num2 <- numero2 + numero1
                      // num2 vale 7
```

Incremento y decremento de variables:

```
num1 <- num1 + 1    // incremento
num2 <- num2 - 1    // decremento
```

Errores frecuentes — CUIDADO

- Asignar a una variable **no definida**
- Usar una variable **sin inicializar**

Asignaciones inválidas

- Entero ← Real (pérdida de decimales)
- Numero ← Cadena (tipos incompatibles)
- Lógico ← cualquier otro (sin conversión)

Asignaciones válidas

- Entero → Real (3 se convierte a 3.0)

Ingreso y Salida de Información en PseInt

Leer — Ingreso de datos

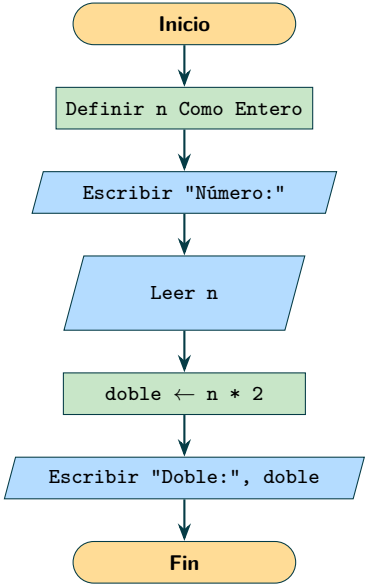
Permite al usuario ingresar un valor que se almacena en una variable.

```
Definir variable Como Entero
Escribir "Ingrese un número: "
Leer variable
```

Escribir — Salida de información

Muestra en pantalla texto y/o valores de variables.

```
Escribir "El valor es: ", variable
Escribir "Suma: ", a+b, " Resta: ", a-b
```



Funciones Matemáticas en PSeInt

Función	Significado
`RC(X)`	Raíz cuadrada de X
`ABS(X)`	Valor absoluto de X
`LN(X)`	Logaritmo natural de X
`EXP(X)`	Función exponencial e^X
`SEN(X)`	Seno de X (*en radianes*)
`COS(X)`	Coseno de X (*en radianes*)
`TAN(X)`	Tangente de X (*en radianes*)
`TRUNC(X)`	Parte entera de X
`REDON(X)`	Entero más cercano a X
`AZAR(X)`	Entero aleatorio en $[0, X-1]$

Notas importantes

- Las funciones trigonométricas reciben el ángulo en **radianes**.
- Para convertir: $\text{rad} = \text{grados} \times \dfrac{\pi}{180}$
- `Aleatorio(n1, n2)`: devuelve un entero aleatorio en el intervalo $[n1, n2]$

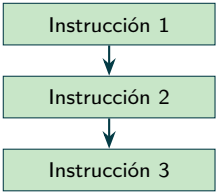
Ejemplos de uso

`RC(16)`	$= 4{,}0$
`ABS(-7)`	$= 7$
`TRUNC(3.9)`	$= 3$
`REDON(3.5)`	$= 4$
`AZAR(6)`	$\in \{0, 1, 2, 3, 4, 5\}$

Estructuras de Control

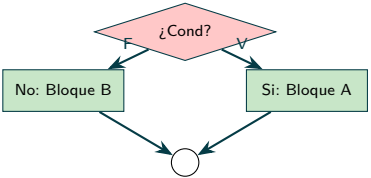
¿Qué son las estructuras de control?

Mecanismos que permiten **controlar el orden** en que se ejecutan las instrucciones de un algoritmo. Son fundamentales en toda programación.



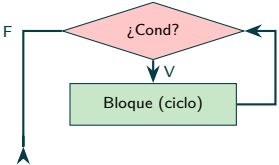
Secuencial

Se ejecutan en orden



Condicional

Toma decisiones



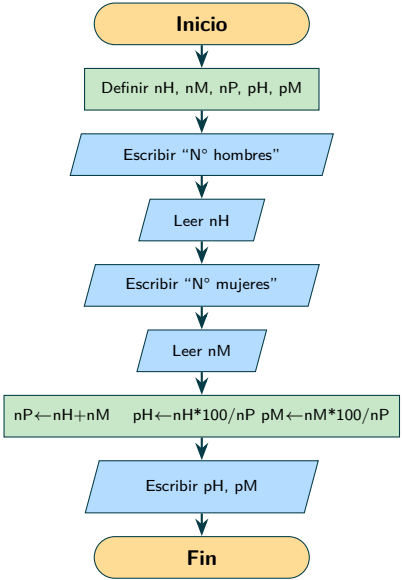
Repetitiva

Repite un bloque

Ejemplo: Estructura Secuencial

Problema: Calcular el porcentaje de hombres y mujeres en un grupo.

```
Algoritmo PorcentajesHM
  Definir nH, nM, nP Como Entero
  Definir pH, pM Como Real

  Escribir "N° de hombres: "
  Leer nH
  Escribir "N° de mujeres: "
  Leer nM

  nP <- nH + nM
  pH <- (nH * 100) / nP
  pM <- (nM * 100) / nP

  Escribir "Hombres: ", pH, " %"
  Escribir "Mujeres: ", pM, " %"
FinAlgoritmo
```

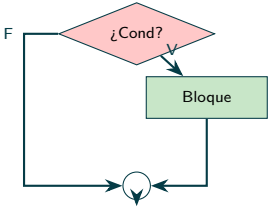


Estructura Condicional — Si / SiNo / FinSi

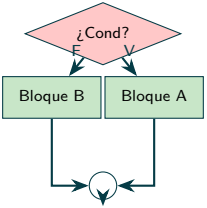
Sintaxis en PSeInt:

```
// Condicional simple
Si <condicion> Entonces
    <instrucciones>
Fin Si

// Condicional doble
Si <condicion> Entonces
    <instrucciones si V>
SiNo
    <instrucciones si F>
Fin Si
```

Simple



Doble



Ejemplo — ¿Par o Impar?

```
Si numero1 % 2 = 0 Entonces
    Escribir "El número es PAR"
SiNo
    Escribir "El número es IMPAR"
Fin Si
```

Regla de oro

La condición siempre produce **Verdadero** o **Falso**. Nunca ambos.

Condicional Múltiple — Segun / Hacer

¿Cuándo usar Segun?

Cuando necesitas evaluar **múltiples casos** de una misma variable entera. Es más limpio que varios Si/SiNo encadenados.

```
Segun <variable> Hacer
    valor1: <instrucciones>
    valor2: <instrucciones>
    valor3, valor4: <instrucciones>
    De Otro Modo:
        <instrucciones por defecto>
Fin Segun
```

Ejemplo — Día de la semana:

```
Algoritmo diaSemana
    Definir N Como Entero
    Escribir "Ingrese 1-7: "
    Leer N
    Segun N Hacer
        1: Escribir "Lunes"
        2: Escribir "Martes"
        3: Escribir "Miércoles"
        4: Escribir "Jueves"
        5: Escribir "Viernes"
        6: Escribir "Sábado"
        7: Escribir "Domingo"
        De Otro Modo:
            Escribir "Rango inválido"
    Fin Segun
FinAlgoritmo
```

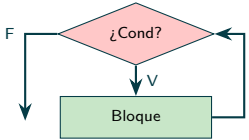
Estructuras Repetitivas — Los Tres Bucles

Mientras

Condición al inicio.

Puede no ejecutarse.

```
Mientras <cond> Hacer
    <instrucciones>
Fin Mientras
```

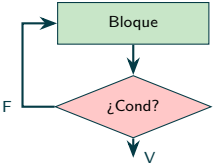


Repetir–Hasta Que

Condición al final.

Ejecuta al menos 1 vez.

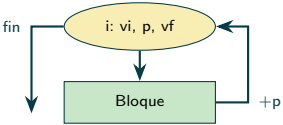
```
Repetir
    <instrucciones>
Hasta Que <cond>
```



Para–Hacer

Número de repeticiones conocido.

```
Para i <- vi Hasta vf
  Con Paso p Hacer
    <instrucciones>
Fin Para
```



Ejemplo — Mostrar números del 1 al 10

`Para i ← 1 Hasta 10 Con Paso 1 Hacer  Escribir i;  Fin Para`

Contadores, Acumuladores e Indicadores

Contador

Variable **entera** que cuenta ocurrencias de un evento.

```
// Inicializar
cont <- 0

// Incrementar
cont <- cont + 1

// (o decrementar)
cont <- cont - 1
```

*Úsalo cuando quieras saber **cuántas veces** ocurrió algo.*

Acumulador

Permite ir sumando (o multiplicando) valores sucesivos.

```
// Inicializar
// suma: en 0
// producto: en 1
acum <- 0

// Acumular
acum <- acum + num
```

*Úsalo cuando quieras **sumar o multiplicar** valores en un ciclo.*

Indicador (bandera)

Variable **lógica** que recuerda si un evento ocurrió.

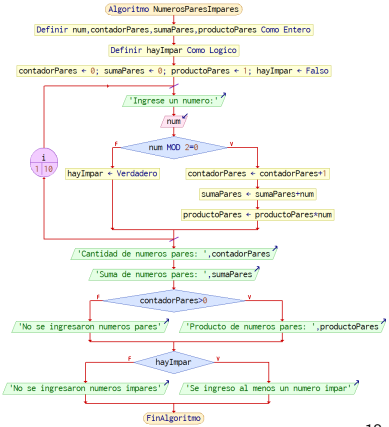
```
// Inicializar
// (no ocurrió aún)
indicador <- Falso

// Cambiar cuando ocurre
indicador <- Verdadero
```

*Úsalo para **recordar** si algo sucedió dentro del ciclo.*

Ejercicio Integrador — Contadores, Acumuladores e Indicadores

Problema: Ingresar 10 números e indicar: (1) cuántos son pares, (2) la suma y el producto de los pares, (3) si se ingresó algún impar.

```
Algoritmo NumerosParesImpares
    Definir num,contadorPares,sumaPares,productoPares Como Entero
    Definir hayImpar Como Logico
    contadorPares ← 0; sumaPares ← 0; productoPares ← 1; hayImpar ← Falso
    Para i←1 Hasta 10 Hacer
        Escribir 'Ingrese un numero:'
        Leer num
        Si num MOD 2=0 Entonces
            contadorPares ← contadorPares+1
            sumaPares ← sumaPares+num
            productoPares ← productoPares*num
        SiNo
            hayImpar ← Verdadero
        FinSi
    FinPara
    Escribir 'Cantidad de numeros pares: ',contadorPares
    Escribir 'Suma de numeros pares: ',sumaPares
    Si contadorPares>0 Entonces
        Escribir 'Producto de numeros pares: ',productoPares
    SiNo
        Escribir 'No se ingresaron numeros pares'
    FinSi
    Si hayImpar Entonces
        Escribir 'Se ingreso al menos un numero impar'
    SiNo
        Escribir 'No se ingresaron numeros impares'
    FinSi
FinAlgoritmo
```



Lectura Complementaria

Arreglos (Arrays) — Estructura de Datos

¿Qué es un arreglo?

Agrupación de **múltiples datos del mismo tipo** bajo un único nombre, accedidos mediante un **índice** (que inicia en 0 o 1 según configuración).

Declaración:

```
Definir vector Como Entero
Dimension vector[10] // vector (1D)

Definir tabla Como Cadena
Dimension tabla[3, 4] // Matriz (2D)
```

Asignar y mostrar:

```
vector[0] <- 10
Escribir vector[0]
tabla[1,1] <- "Hola"; Escribir tabla[1,1]
```

Visualización de un vector:

vector	10	25	7	42	3
	[0]	[1]	[2]	[3]	[4]

Recorrer con bucle:

```
Para i <- 0 Hasta 9 Hacer
    Escribir vector[i]
Fin Para
```

Ventaja clave

Sin arreglos, necesitarías declarar num0, num1, . . . , num9 por separado. ¡10 variables vs 1 arreglo!

Funciones en PSeInt

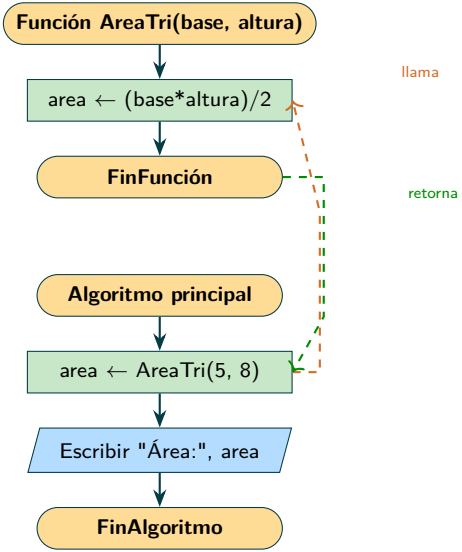
¿Qué es una función?

Bloque de código **reutilizable** que realiza una tarea específica y puede devolver un resultado al programa principal.

```
Funcion resultado <- Nombre(arg1, arg2)
    // instrucciones
    resultado <- <expresión>
FinFuncion
```

Ejemplo — Área del triángulo:

```
Funcion area <- AreaTri(base, altura)
    area <- (base * altura) / 2
FinFuncion
Algoritmo principal
    area <- AreaTri(5, 8)
    Escribir "Área: ", area
FinAlgoritmo
```



Ventajas de Usar Funciones

Reutilización de código

Escribe la función una vez, llámala cuantas veces necesites. Evita duplicar código.

Modularidad

Divide el programa en piezas pequeñas y manejables. Cada función hace una sola cosa bien.

Facilita la depuración

Puedes probar cada función por separado antes de integrarla. Los errores son más fáciles de localizar.

Mejora la legibilidad

Un nombre descriptivo como `CalcularPromedio()` comunica claramente qué hace el código.

Flexibilidad

Cambia la implementación interna de una función sin afectar el resto del programa.

Principio clave

Una función = Una responsabilidad

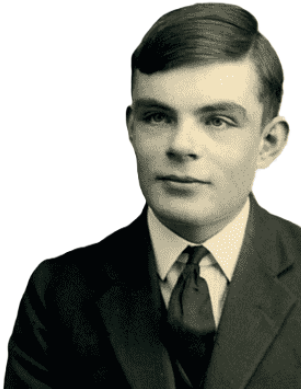
Si una función hace demasiadas cosas, divídela.

Próxima sesión

Variables y estructuras secuenciales en C++

achulluncuy@uni.edu.pe

Repositorio del curso

Aula virtual

"A computer would deserve to be called intelligent if it could deceive a human into believing that it was human." — Alan Turing